

Universidad Técnica Estatal de Quevedo

Facultad de Ciencias de la Computación

Carrera de Software

Proyecto de Fin de Carrera — Aplicaciones Web

Periodo académico: 2025 – 2026 — Curso/Paralelo: A

BIOPET

Sistema web de gestión veterinaria

Especificación de Requisitos de Software (SRS) — v1.0.0

Docente-director: Guerrero Ulloa Gleiston Ciceron

Equipo: BMT

Integrantes

Fred Adrián Beltrán Montiel	fbeltranm@uteq.edu.ec	ORCID: 0009-0007-8303-2137
Jaime Josué Mariscal Cabrera	jmariscalc@uteq.edu.ec	ORCID: 0009-0007-2206-3941
Zaida Melissa Taipe Mora	ztaipem@uteq.edu.ec	ORCID: 0009-0000-1227-3258

Repositorio: <https://github.com/Grinjoww/ENTREGA-FINAL-BIOPET>

Quevedo, 7 de septiembre de 2026

Índice

Especificación de Requisitos de Software (SRS) — BIOPET	1
Índice	1
1. Introducción	2
1.1. Propósito	2
1.2. Alcance	2
1.3. Definiciones, acrónimos y abreviaturas	2
1.4. Referencias	3
1.5. Resumen del documento	3
2. Descripción global	3
2.1. Perspectiva del producto	3
2.2. Cambio de plataforma tecnológica respecto a la Entrega 1A	3
2.3. Funciones del producto (resumen)	3
2.4. Características de los usuarios	4
2.5. Restricciones	4
2.6. Supuestos y dependencias	4
3. Requisitos específicos	4
3.1. Requisitos funcionales (REQ-F)	4
3.2. Requisitos no funcionales (REQ-NF)	11
3.3. Criterios INVEST aplicados a las historias de usuario	13
4. Trazabilidad (resumen)	15
4.1. Trazabilidad histórica: identificadores originales → identificadores actuales (cierre de OBS-03)	16
5. Modelo de datos (referencia)	17
6. Interfaces de usuario (referencia)	18
7. Observaciones e información pendiente	18
APROBACIÓN DEL DOCENTE-DIRECTOR	20

Especificación de Requisitos de Software (SRS) — BIOPET

Versión: v1.0.0 (Entrega Final, PFC Aplicaciones Web 2026-2027) **Conforme a:** ISO/IEC/IEEE 29148:2018 (estructura de SRS), INCOSE Guide to Writing Requirements v4 (calidad de requisitos individuales y de conjunto), criterios INVEST de Cohn (historias de usuario, aplicados explícitamente en la sección 3.3), plantilla de Cockburn (casos de uso).

Estado de aprobación: documento preparado para revisión y aprobación del docente-director. Esta copia (SRS-v1.0.0-para-firma.pdf) se envía específicamente para su firma electrónica; el estado de firma se actualizará en el documento histórico (SRS-v1.0.0.pdf), con fecha real, únicamente cuando exista una firma legítima recibida del docente — nunca antes, y nunca simulada o copiada de otro documento.

Procedencia de este documento: este archivo se reconstruye a partir de dos fuentes: (1) el SRS de la Entrega 1A (PFC_Entrega1A_BMT.pdf), que especificaba el backend sobre ASP.NET Core 8/C#, y (2) el código realmente implementado en las Entregas 1B y Tercera Entrega, sobre **Java 21 + Spring Boot 3.2 + Spring Security 6 + Spring Data JPA + PostgreSQL 16 + Redis 7**. Ningún requisito de este documento fue inventado sin sustento: cada uno proviene del SRS original, del código fuente verificado, o de la Guía oficial de la Tercera Entrega. El detalle de cada cambio respecto al documento original está en docs/requisitos/cambios/CAMBIOS-SRS.md.

Índice

- 1. Introducción
- 2. Descripción global

3. Requisitos específicos 3.1. Requisitos funcionales (REQ-F) 3.2. Requisitos no funcionales (REQ-NF) 3.3. Criterios INVEST aplicados a las historias de usuario
 4. Trazabilidad (resumen)
 5. Modelo de datos (referencia)
 6. Interfaces de usuario (referencia)
 7. Observaciones e información pendiente
-

1. Introducción

1.1. Propósito

Este documento especifica los requisitos funcionales y no funcionales del sistema BIOPET en su estado **v1.0.0 (Entrega Final)**. Su propósito es servir como fuente única de verdad para la matriz de trazabilidad, las pruebas automatizadas y la evidencia empírica exigidas por el bloque A.3 de la Guía, incorporando además los módulos de la Unidad IV (citas, consultas, vacunas, gestión administrativa de usuarios y consulta externa de especies) que no existían en la revisión v0.9.0-rc de la Tercera Entrega.

1.2. Alcance

BIOPET es un sistema web para centralizar la información clínica y administrativa de clínicas veterinarias de pequeña y mediana escala. El alcance completo del producto (definido en la Entrega 1A) contempla gestión de dueños y mascotas, historial clínico, citas, telemetría IoT, recomendación clínica asistida, facturación digital y reportes.

Alcance implementado y verificado en v1.0.0: autenticación completa (registro, login, refresh, logout con revocación), control de acceso por rol (RBAC), CRUD completo de la entidad Mascota con verificación de propiedad, y resumen agregado de mascotas por especie (vía función SQL). Este es el subconjunto Must Have que el equipo se comprometió a entregar operativo y reproducible desde la Tercera Entrega (v0.9.0-rc).

Ampliado en v1.0.0 (Unidad IV): gestión administrativa de usuarios (REQ-F-023), CRUD de vacunas (REQ-F-024) y consulta externa de información de especies con caché (REQ-F-025), los tres con interfaz de usuario real para vacunas (`frontend/src/app/features/vacunas.component.ts`) y solo a nivel de API para usuarios/especies externas (sin pantalla propia todavía). Adicionalmente, el backend incorporó CRUD completo de citas (`CitaController`) y de consultas médicas (`ConsultaController`), que formalizan parcialmente REQ-F-015 y REQ-F-013 respectivamente (detalle de alcance real, sin sobre-declarar, en la sección 3.1).

Alcance pendiente, heredado de la Entrega 1A: vista consolidada de historial clínico cronológico y calendario interactivo de citas (ambos con backend real desde v1.0.0, sin la pieza específica pendiente — ver “Ampliado en v1.0.0” arriba y la sección 3.1), más prescripción de medicamentos, telemetría IoT, ubicación en mapa, recomendaciones asistidas, facturación digital y reportes exportables (estos últimos seis, sin ningún código de backend ni de frontend todavía). El modelo de datos conceptual de estos módulos ya existe (ver sección 5 y el DER de la Entrega 1A). Se documentan como requisitos con estado “pendiente” o “parcial” (según corresponda) para que la matriz de trazabilidad (bloque A.3.3 de la Guía) los declare correctamente, en vez de omitirlos.

1.3. Definiciones, acrónimos y abreviaturas

Término	Definición
RBAC	Role-Based Access Control: control de acceso basado en el rol del usuario autenticado.
JWT	JSON Web Token (RFC 7519): token firmado que representa afirmaciones sobre un sujeto autenticado.
ORM	Object-Relational Mapping: mapeo objeto-relacional (Spring Data JPA / Hibernate en este proyecto).

Término	Definición
SP	Stored Procedure / función almacenada en el motor de base de datos.
TTL	Time To Live: tiempo de vida de una entrada de caché.
MoSCoW	Técnica de priorización: Must, Should, Could, Won't.
shall	Verbo modal normativo de ISO/IEC/IEEE 29148 que indica obligación contractual del requisito.

1.4. Referencias

- ISO/IEC/IEEE 29148:2018 — Requirements Engineering.
- INCOSE Guide to Writing Requirements v4.
- RFC 7519 (JWT), RFC 7807 (ProblemDetails).
- Guía de la Tercera Entrega — PFC Aplicaciones Web 2026-2027, UTEQ.
- PFC_Entrega1A_BMT.pdf — SRS y diseño original (ASP.NET Core).
- docs/requisitos/cambios/CAMBIOS-SRS.md — registro de cambios respecto al documento original.

1.5. Resumen del documento

La sección 2 describe el producto de forma global (perspectiva, funciones, usuarios, restricciones). La sección 3 contiene el detalle de cada requisito funcional y no funcional con su identificador persistente, prioridad MoSCoW, criterio de aceptación y método de verificación. La sección 4 resume la trazabilidad end-to-end. Las secciones 5 y 6 remiten a los artefactos de modelo de datos e interfaz que ya existen en el repositorio. La sección 7 documenta, sin inventar contenido, lo que aún falta por completar.

2. Descripción global

2.1. Perspectiva del producto

BIOPET es un sistema nuevo, no una extensión de un producto previo existente en la clínica. Reemplaza procesos manuales (hojas de cálculo, mensajería) por una plataforma web centralizada, según el problema documentado en la Entrega 1A (entrevistas a tres veterinarios y dos auxiliares, encuestas a quince dueños de mascotas).

2.2. Cambio de plataforma tecnológica respecto a la Entrega 1A

El SRS original (Entrega 1A) especificaba ASP.NET Core 8/C# como backend (ADR-001 original) y Bootstrap 5 + HTML/CSS/JS como frontend. El equipo migró la implementación real a **Java 21 + Spring Boot 3.2** en el backend y **Angular 17+** en el frontend, decisión reflejada en ADR-002-pila-tecnologica.md del repositorio actual. Los requisitos funcionales de negocio (qué hace el sistema) no cambiaron; los requisitos no funcionales técnicos (cómo se implementa: framework, ORM, mecanismo JWT) se actualizaron para reflejar la pila real, evitando que el SRS describa un sistema que no es el que existe en el repositorio.

2.3. Funciones del producto (resumen)

- Gestión de usuarios y autenticación (roles: ADMIN, VETERINARIO, AUXILIAR, DUENO), incluida la administración de cuentas por un ADMIN (Unidad IV).
- Gestión de mascotas (CRUD + resumen agregado por especie).
- Gestión de vacunas (CRUD, con interfaz de usuario) y de citas y consultas médicas (CRUD a nivel de API; sin calendario interactivo ni vista de historial clínico consolidado todavía — ver 3.1).
- Consulta de información externa de especies (taxonomía, hábitat, dieta), con caché.

- (*Pendiente*) Vista consolidada de historial clínico cronológico, prescripción de medicamentos, telemetría IoT, recomendaciones asistidas, facturación y reportes exportables.

2.4. Características de los usuarios

Rol	Descripción	Nivel técnico esperado
ADMIN	Supervisa roles, operatividad general del sistema y tiene visibilidad global de datos.	Medio-alto (personal administrativo de la clínica).
VETERINARIO	Registra diagnósticos y gestiona historiales clínicos (módulo pendiente); gestiona mascotas.	Medio (personal clínico).
AUXILIAR	Apoya operaciones administrativas y de gestión de mascotas.	Medio.
DUENO	Consulta únicamente la información de sus propias mascotas.	Básico (público general, sin capacitación previa).

2.5. Restricciones

- El proyecto debe completarse dentro de un semestre académico (PPA 2026-2027), lo que limita el alcance implementado al núcleo Must Have.
- La base de datos debe ser PostgreSQL 16 (decisión ya tomada, ver `ADR-004-postgresql.md`).
- La autenticación debe usar JWT en cookies `HttpOnly + Secure + SameSite=Strict` (bloque A.1 de la Guía), no `localStorage` ni `sessionStorage`.
- Toda operación de base de datos que no sea CRUD elemental debe implementarse como función/procedimiento almacenado (bloque A.2 de la Guía), no como JPQL/HQL con joins o agregaciones.

2.6. Supuestos y dependencias

- Se asume disponibilidad de un motor PostgreSQL 16 y Redis 7 accesibles desde el backend (vía Docker Compose en desarrollo).
- Se asume que los servicios externos mencionados en la Entrega 1A (IoT, IA cognitiva, correo) se integrarán en fases posteriores; ninguno está implementado todavía y no se documentan requisitos técnicos de integración hasta que exista una decisión de arquitectura formal para ellos.

3. Requisitos específicos

Cada requisito seguirá el patrón [condición] [sujeto] shall [acción] [objeto] [restricción] de ISO/IEC/IEEE 29148, con identificador persistente, rationale, prioridad MoSCoW, criterio de aceptación medible y método de verificación, cumpliendo las características INCOSE C1–C9 (Necessary, Appropriate, Unambiguous, Complete, Singular, Feasible, Verifiable, Correct, Conforming).

3.1. Requisitos funcionales (REQ-F)

Nota de consistencia (corrección respecto a una versión anterior de este documento): la numeración de esta sección es la que ya está fijada en `docs/requisitos/historias/HistoriasUsuario.md` (HU-001 a HU-020) y `docs/requisitos/casos-de-uso/CasosDeUso.md` (CU-01 a CU-20), documentos que ya existían en el repositorio con 20 requisitos funcionales completos (incluyendo uno propio para “consultar mascota por id” y otro para “auditoría”, que una reconstrucción anterior de este SRS no tenía desglosados igual). Esta versión del SRS adopta esa numeración como única fuente de verdad, para que los tres documentos no se contradigan entre sí.

Implementados y verificados (REQ-F-001 a REQ-F-012, REQ-F-021) **REQ-F-001 — Registro de usuario dueño de mascota** - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir que cualquier visitante se registre proporcionando nombre, correo electrónico y contraseña, asignándole automáticamente el rol `ROLE_DUEÑO`, ignorando cualquier rol enviado por el cliente. - **Rationale:** heredado de RF-01 de la Entrega 1A; el registro público solo aplica al rol dueño, los demás roles se crean administrativamente. - **Verificación:** `Test AuthControllerTest` (registro exitoso). - **Trazabilidad:** → HU-001 → CU-01 → `AuthController.registro` → `AuthService.registrar` → POST `/api/auth/registro`. - **Estado:** verificado.

REQ-F-002 — Rechazo de registro con correo duplicado - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** Al recibir una solicitud de registro con un correo electrónico ya existente, el sistema deberá rechazarla con un código 409 Conflict y un cuerpo `ProblemDetails`, sin crear ningún registro. - **Rationale:** funcionalidad presente en el código (`EmailDuplicadoException`) no documentada como requisito independiente en el SRS original de la Entrega 1A; se agrega para cerrar el hueco (bloque A.3 de la Guía). - **Verificación:** `Test AuthControllerTest` (registro con correo duplicado). - **Trazabilidad:** → HU-001 → CU-01 → `AuthService.registrar` → `EmailDuplicadoException` → `GlobalExceptionHandler`. - **Estado:** verificado.

REQ-F-003 — Autenticación mediante usuario y contraseña - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir que un usuario registrado inicie sesión mediante correo electrónico y contraseña, emitiendo un access token (1 hora de vigencia) y un refresh token con los siete claims JWT estándar. - **Rationale:** heredado de RF-16/RF-WEB-01 de la Entrega 1A. - **Verificación:** `Test AuthControllerTest`, `JwtServiceTest`. - **Trazabilidad:** → HU-002 → CU-02 → `AuthController.login` → POST `/api/auth/login`. - **Estado:** verificado.

REQ-F-004 — Renovación de sesión (refresh) - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir renovar el access token utilizando un refresh token válido y no revocado, sin exigir nuevamente las credenciales del usuario. - **Rationale:** heredado de RNF-03/RNF-WEB-03 de la Entrega 1A (expiración configurable de JWT), llevado a requisito funcional explícito. - **Verificación:** `Test AuthControllerTest` (flujo refresh). - **Trazabilidad:** → HU-003 → CU-03 → `AuthController.refresh` → POST `/api/auth/refresh`. - **Estado:** verificado.

REQ-F-005 — Cierre de sesión con revocación de token - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir cerrar sesión, registrando el `jwt` del token vigente en una lista negra en Redis con TTL igual a su tiempo restante de expiración, de modo que una solicitud posterior con ese token a un recurso protegido responda 401. - **Rationale:** heredado de RF-17/RF-WEB-04 de la Entrega 1A; mecanismo de revocación definido en ADR-003-jwt-redis.md. - **Verificación:** `Test AuthControllerTest`, `AuthenticationAuditServiceTest`. - **Trazabilidad:** → HU-004 → CU-04 → `AuthController.logout` → `TokenBlacklistService.revoke` → POST `/api/auth/logout`. - **Estado:** verificado.

REQ-F-006 — Control de acceso por rol (RBAC) - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá restringir el acceso a cada endpoint protegido según el rol del usuario autenticado, rechazando con 403 Forbidden cualquier solicitud de un rol no autorizado para ese recurso. - **Rationale:** heredado de RF-13/RF-WEB-02 de la Entrega 1A; caso de uso transversal incluido implícitamente por la gestión de mascotas. - **Verificación:** `Test MascotaControllerTest` (casos 403 por rol). - **Trazabilidad:** → HU-005 → CU-05 → anotaciones `@PreAuthorize` en `MascotaController`. - **Estado:** verificado.

REQ-F-007 — Consulta del perfil propio - **Tipo:** Funcional · **Prioridad:** Should - **Enunciado:** El sistema deberá permitir que un usuario autenticado consulte sus propios datos de perfil (nombre, correo, rol), sin exponer el listado completo de usuarios. - **Rationale:** funcionalidad presente en el código (GET `/api/usuarios/me`), base del `authGuard` de Angular; no documentada como requisito independiente en el SRS original. - **Verificación:** `Test UsuarioControllerTest` (prueba `meSigueFuncionando` sobre GET `/api/usuarios/me`); inspección de `UsuarioController`. - **Trazabilidad:** → HU-006 → CU-06 → `UsuarioController.me` → `AuthService.perfil`. - **Estado:** verificado.

REQ-F-008 — Creación de mascota - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir que un usuario con rol ADMIN, VETERINARIO o AUXILIAR registre una nueva mascota (nombre, especie, raza, fecha de nacimiento) asociada a un dueño existente con rol `ROLE_DUEÑO`. - **Rationale:** heredado

de RF-01/RF-02 de la Entrega 1A (registro y asociación con propietario), acotado a la entidad Mascota implementada en esta fase. - **Verificación:** Test MascotaControllerTest (creación exitosa y validación de campos). - **Trazabilidad:** → HU-007 → CU-07 → MascotaController.crear → POST /api/mascotas. - **Estado:** verificado.

REQ-F-009 — Listado paginado de mascotas activas, con filtrado por propietario según rol - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir consultar el listado paginado de mascotas activas; si el solicitante tiene rol ROLE_DUEÑO, el listado deberá restringirse únicamente a sus propias mascotas; para los demás roles autorizados, el listado deberá incluir todas las mascotas activas. - **Rationale:** heredado de RF-02 de la Entrega 1A, con la regla de aislamiento de datos entre dueños explícita en el código (MascotaService.listar). - **Verificación:** Test MascotaControllerTest (listado por rol ROLE_DUEÑO vs. resto). - **Trazabilidad:** → HU-008 → CU-08 → MascotaController.listar → GET /api/mascotas. - **Estado:** verificado.

REQ-F-010 — Consulta de mascota por identificador - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir consultar el detalle completo de una mascota a partir de su identificador, respondiendo 404 con ProblemDetails si no existe. - **Rationale:** complementa REQ-F-009 para el caso de consulta puntual, necesario antes de una edición o de mostrar el detalle en la interfaz; no estaba desglosado como requisito independiente en una versión anterior de este documento. - **Verificación:** Test MascotaControllerTest y RecursoNoEncontradoException. - **Trazabilidad:** → HU-009 → CU-09 → MascotaController.buscar → GET /api/mascotas/{id}. - **Estado:** verificado.

REQ-F-011 — Actualización de mascota con verificación de propiedad - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir actualizar los datos de una mascota existente a un usuario con rol ADMIN, VETERINARIO o AUXILIAR, registrando la fecha de actualización (actualizado_en). - **Rationale:** heredado de RF-02 de la Entrega 1A. - **Verificación:** Test MascotaControllerTest (actualización exitosa y caso 403/404). - **Trazabilidad:** → HU-010 → CU-10 → MascotaController.actualizar → PUT /api/mascotas/{id}. - **Estado:** verificado.

REQ-F-012 — Baja lógica de una mascota - **Tipo:** Funcional · **Prioridad:** Must - **Enunciado:** El sistema deberá permitir dar de baja lógicamente (cambio del atributo activo a false) una mascota existente, sin eliminar físicamente el registro de la base de datos. - **Rationale:** preservación de historial e integridad referencial futura con historial clínico y citas (módulos pendientes). - **Verificación:** Test MascotaControllerTest (baja y verificación de que la mascota deja de aparecer en el listado activo). - **Trazabilidad:** → HU-011 → CU-11 → MascotaController.eliminar → DELETE /api/mascotas/{id}. - **Estado:** verificado.

REQ-F-021 — Resumen de mascotas activas agrupadas por especie - **Tipo:** Funcional · **Prioridad:** Should - **Enunciado:** El sistema deberá permitir consultar el total de mascotas activas agrupadas por especie; para roles distintos de ADMIN, el resultado deberá restringirse siempre a las mascotas del propio usuario autenticado; para el rol ADMIN, el resultado deberá poder filtrarse por un dueño específico o consultarse sin filtro para obtener el total global. - **Rationale:** nuevo requisito de la Tercera Entrega, exigido por el bloque A.2.2 de la Guía como ejemplo de operación agregada (GROUP BY) que debe implementarse obligatoriamente vía función/procedimiento almacenado, no vía JPQL. Se numera 021 (no 013) porque los identificadores 013 a 020 ya estaban ocupados por los requisitos heredados de la Entrega 1A, fijados en HistoriasUsuario.md/CasosDeUso.md antes de que este requisito existiera. - **Verificación:** Test de integración ResumenEspeciesIntegrationTest (Testcontainers, PostgreSQL real). - **Trazabilidad:** → HU-020 → CU-20 → MascotaController.resumenPorEspecies → MascotaService.resumenPorEspecie → MascotaRepository.resumenPorEspecie → función fn_resumen_mascotas_por_especie → GET /api/mascotas/resumen-especies. - **Tipo de acceso a datos:** SP (agregación con GROUP BY, prohibido en ORM elemental por el bloque A.2.1 de la Guía). - **Estado:** verificado.

Pendientes, heredados de la Entrega 1A (REQ-F-013 a REQ-F-020)

Estos requisitos provienen directamente de RF-03 a RF-15 del SRS original y están desglosados exactamente como en HistoriasUsuario.md (HU-012 a HU-019) y CasosDeUso.md (CU-12 a CU-19). El modelo de datos conceptual ya existe en el DER de la Entrega 1A (entidades Cita,

Historial_Clinico, Dispositivo_IoT, Chat_Triage, Producto_Servicio, Factura, etc.), pero **no hay código de backend ni de frontend implementado** para ninguno de ellos en v0.9.0-rc. Se reclasifican de prioridad Must/Media (documento original) a Should/Could, porque no forman parte del compromiso de esta entrega, y se marcan como “pendiente” para que la matriz de trazabilidad los declare sin ocultarlos.

Id	Descripción (resumen)	Prioridad	Origen	HU / CU	Estado
REQ-F-013	Registrar atención médica y almacenar/consultar el historial clínico de cada mascota de forma cronológica.	Should	RF-03, RF-04	HU-012 / CU-12	parcial
REQ-F-014	Registrar medicamentos prescritos durante una atención médica.	Could	RF-05	HU-013 / CU-13	pendiente
REQ-F-015	Registrar, modificar y consultar citas veterinarias mediante calendario interactivo.	Should	RF-06	HU-014 / CU-14	parcial
REQ-F-016	Proveer una API para recibir datos de dispositivos IoT de rastreo asociados a mascotas.	Could	RF-08, RF-09	HU-015 / CU-15	pendiente
REQ-F-017	Generar recomendaciones clínicas informativas a partir del historial médico (enunciado ampliado — ver bloque detallado, cierre de OBS-04).	Could	RF-10	HU-016 / CU-16	pendiente

Id	Descripción (resumen)	Prioridad	Origen	HU / CU	Estado
REQ-F-018	Generar comprobantes de pago digitales en PDF y registrar el método de pago de un servicio veterinario.	Should	RF-11, RF-12	HU-017 / CU-17	pendiente
REQ-F-019	Generar reportes estadísticos exportables en PDF y Excel.	Could	RF-14	HU-018 / CU-18	pendiente
REQ-F-020	Registrar las operaciones relevantes del sistema (usuario, fecha, hora) para fines de auditoría.	Should	RF-15	HU-019 / CU-19	parcial
REQ-F-022	Enviar notificaciones al usuario por correo electrónico ante eventos relevantes de su cuenta.	Could	RF-07	HU-021 / CU-21	pendiente

Nota sobre REQ-F-013 y REQ-F-015 (revisión v1.0.0 — código actual): a diferencia de los demás requisitos de esta tabla, no se dejan como “pendiente” en v1.0.0 porque la Unidad IV sí incorporó código real para ambos, verificado contra el repositorio actual (paso de revisión obligatorio antes de cerrar esta entrega): `CitaController/ CitaService/CitaRepository` (CRUD completo: listar, buscar, crear, actualizar, eliminar) y `ConsultaController/ConsultaService/ConsultaRepository` (mismo CRUD), ambos con clase de test dedicada (`CitaControllerTest`, `ConsultaControllerTest`). Se marcan “**parcial**”, no “verificado”, porque el enunciado original exige más de lo que el CRUD por sí solo cubre: REQ-F-015 pide explícitamente un “calendario interactivo” (no existe pantalla de citas en `frontend/src/app/features/`, solo la API) y REQ-F-013 pide el historial clínico “de forma cronológica” (existe la proyección `HistorialClinico`/función `fn_historial_clinico_mascota` a nivel de repositorio, pero ningún controlador la expone todavía como endpoint consolidado — la única vía real hoy es el CRUD genérico de consultas, sin agrupar ni ordenar por mascota). `docs/trazabilidad/matriz.csv` registra además, para REQ-F-013, que `ConsultaService.listar()` **no filtra por propietario para ROLE_DUEÑO** pese a la regla general de aislamiento de datos que sí aplican `MascotaService/VacunaService` (ver REQ-F-009); y que `GET /api/consultas (listado)` y `PUT /api/consultas/{id}` no tienen prueba automatizada dedicada en `ConsultaControllerTest`. Ambos quedan como limitación explícita en la sección 7, no como pendientes ocultos ni como “verificado” sin serlo del todo.

Nota sobre REQ-F-020 (auditoría): a diferencia de los demás requisitos de esta tabla, no está 100 % pendiente: el registro de eventos de *autenticación* (login exitoso/fallido, bloqueo por rate limit, refresh, logout) ya opera en producción y está cubierto por REQ-NF-009

(`AuthenticationAuditService`). Lo que falta es la auditoría genérica de operaciones CRUD sobre mascotas y los módulos pendientes. Se marca “parcial”, no “pendiente”, para no ocultar el trabajo ya hecho.

Nota sobre REQ-F-017 (cierre de OBS-04): la retroalimentación oficial del SGA de la Entrega 1A señaló, como observación leve, la ambigüedad de la redacción original de RF-10 (“recomendaciones informativas”; ver `docs/observaciones/OBSERVACIONES.md`, OBS-04). El identificador y el alcance no cambian (sigue siendo REQ-F-017, prioridad `Could`, estado `pendiente`, dependiente de REQ-F-013); lo que cambia es exclusivamente la redacción, reformulada en patrón “El sistema deberá...” con entradas, resultado esperado y criterios de aceptación verificables, sin ampliar la funcionalidad prevista originalmente. El detalle completo reemplaza al resumen de una sola línea que tenía esta fila en versiones anteriores del SRS.

REQ-F-017 — Generación de recomendaciones clínicas informativas a partir del historial médico - Tipo: Funcional · **Prioridad:** `Could` - **Enunciado:** Al recibir una solicitud de recomendaciones para una mascota con historial clínico registrado, el sistema deberá generar, mediante un servicio de IA externo, una lista de recomendaciones de cuidado en texto a partir de los datos del historial clínico de esa mascota, y deberá devolver cada recomendación acompañada de la advertencia explícita “informativa, no sustituye diagnóstico veterinario”. - **Entradas:** identificador de una mascota con historial clínico registrado (depende de REQ-F-013, módulo pendiente). - **Resultado esperado:** una lista de cero a N recomendaciones en texto plano, cada una acompañada de la advertencia de carácter informativo; si la mascota no tiene historial clínico registrado, el sistema deberá responder con una lista vacía, no con un error. - **Criterios de aceptación (verificables):** 1. Dado un historial clínico existente para la mascota solicitada, cuando se invoca el endpoint de recomendaciones, entonces la respuesta incluye el campo `recomendaciones: string[]` y, por cada elemento, el campo `advertencia: "informativa, no sustituye diagnóstico veterinario"`. 2. Dado que la mascota no tiene historial clínico registrado, cuando se solicita el endpoint de recomendaciones, entonces la respuesta es `200 OK` con `recomendaciones: []`. 3. El proveedor concreto del servicio de IA es una decisión de arquitectura pendiente; no se implementa en v0.9.0-rc. - **Rationale:** heredado de RF-10 de la Entrega 1A; reformulado en la Tercera Entrega para cerrar OBS-04 (ambigüedad leve señalada por el docente en “recomendaciones informativas”), sin ampliar el alcance original: sigue dependiendo de REQ-F-013 (historial clínico, módulo pendiente) y del mismo servicio de IA externo ya previsto desde la Entrega 1A. - **Verificación:** pendiente — no implementado en v0.9.0-rc (depende de REQ-F-013). Método de verificación previsto: test de integración con un doble de prueba (mock) del servicio de IA, validando el contrato de entrada/salida y el caso sin historial clínico. - **Trazabilidad:** → HU-016 → CU-16 → (módulo pendiente; depende de REQ-F-013). - **Estado:** pendiente (redacción cerrada; implementación no iniciada).

Nota sobre REQ-F-022 (cierre de OBS-02): este requisito no forma parte de la secuencia original RF-03 a RF-15; se numera 022 (siguiente identificador libre después de REQ-F-021) porque corresponde a **RF-07**, el requisito que la retroalimentación oficial del SGA de la Entrega 1A señaló como ausente de la lista consolidada (“FALTA RF-07 en la lista consolidada, salta RF-06 -> RF-08”; ver `docs/observaciones/OBSERVACIONES.md`, OBS-02). RF-07 no estaba perdido: el diagrama de contexto C4 Nivel 1 (`docs/diagrams/c4-contexto/C4-L1-contexto.md`, sección “Trazabilidad”) ya lo identificaba como el “Servicio de Correos” mencionado en la Entrega 1A, sin que ninguna versión anterior de este SRS le hubiera asignado un identificador **REQ-F** propio. Se le asigna aquí REQ-F-022 (no REQ-F-007, que ya está ocupado por un requisito distinto — “Consulta del perfil propio” — para no duplicar identificadores) con su enunciado completo, criterios de aceptación verificables y trazabilidad propia (ver bloque detallado más abajo).

REQ-F-022 — Notificaciones al usuario por correo electrónico (Servicio de Correos, RF-07 recuperado) - Tipo: Funcional · **Prioridad:** `Could` - **Enunciado:** Al ocurrir un evento relevante para la cuenta de un usuario (por ejemplo, registro exitoso), el sistema deberá enviar una notificación por correo electrónico al usuario afectado a través de un servicio de correo externo, de forma asíncrona, sin bloquear ni condicionar la respuesta HTTP de la operación que originó el evento. - **Entradas:** un evento de dominio notificable (tipo de evento, correo electrónico destinatario, datos contextuales mínimos del evento). - **Resultado esperado:** el correo se envía o se encola para envío de forma asíncrona; un fallo del servicio de correo externo no debe revertir ni hacer fallar la operación de dominio que originó el evento. - **Criterios**

de aceptación (verificables): 1. Dado un registro de usuario exitoso, cuando el servicio de correo está disponible, entonces se envía un correo de bienvenida a la dirección registrada. 2. Dado que el servicio de correo externo no está disponible, cuando ocurre un evento notificable, entonces la operación de dominio que lo originó (por ejemplo, el registro) completa exitosamente y el fallo de envío queda registrado en el log del sistema, sin propagarse como error al cliente. 3. El proveedor de correo concreto (SMTP propio, SES, SendGrid u otro) es una decisión de arquitectura pendiente; no está implementado en v0.9.0-rc. - **Rationale:** corresponde al RF-07 original de la Entrega 1A (“Servicio de Correos”), documentado como sistema externo en `docs/diagrams/c4-contexto/C4-L1-contexto.md` pero sin requisito REQ-F formal hasta esta revisión. Se incorpora explícitamente para cerrar OBS-02 sin duplicar REQ-F-007 (funcionalidad distinta ya implementada). - **Verificación:** pendiente — no implementado en v0.9.0-rc. Método de verificación previsto: test de integración con un servidor SMTP de pruebas (por ejemplo, Mailhog) que confirme el envío en el caso exitoso y el comportamiento no bloqueante ante fallo del servicio externo. - **Trazabilidad:** → HU-021 → CU-21 → (módulo pendiente; sin controlador ni servicio implementados en el backend). - **Estado:** pendiente.

Incorporados en Unidad IV (REQ-F-023 a REQ-F-025)

Nota de alcance: los tres requisitos siguientes formalizan funcionalidades reales incorporadas al backend durante la Unidad IV (Citas y Consultas ya tenían requisito formal propio — REQ-F-015 y REQ-F-013 respectivamente — y no se duplican aquí). Cada uno agrupa el módulo completo, no una operación HTTP individual: la trazabilidad hacia cada endpoint real se detalla en `docs/trazabilidad/matriz.csv`.

REQ-F-023 — Gestión administrativa de usuarios - Tipo: Funcional · **Prioridad:** Should - **Enunciado:** El sistema deberá permitir que un usuario con rol ADMIN liste, consulte por identificador, cree, actualice y dé de baja lógica cuentas de usuario de cualquier rol, sin permitir que un administrador modifique el rol de su propia cuenta. - **Rationale:** nuevo requisito de la actividad de Unidad IV; formaliza el CRUD administrativo de `UsuarioController/UsuarioService`, funcionalidad distinta de REQ-F-007 (consulta del perfil propio, `/api/usuarios/me`), que no se modifica ni se duplica. Se numera 023 (siguiente identificador libre después de REQ-F-022). - **Verificación:** Test `UsuarioControllerTest` (16 pruebas: listado, consulta, creación, actualización, baja lógica, correo duplicado, contraseña obligatoria, no autoescalada de rol propio, control de acceso por rol). - **Trazabilidad:** → HU-022 → CU-22 → `UsuarioController.{listar,buscar,crear,actualizar,eliminar}` → `UsuarioService` → GET `/api/usuarios`, GET `/api/usuarios/{id}`, POST `/api/usuarios`, PUT `/api/usuarios/{id}`, DELETE `/api/usuarios/{id}`. - **Estado:** verificado.

REQ-F-024 — Gestión de vacunas - Tipo: Funcional · **Prioridad:** Should - **Enunciado:** El sistema deberá permitir registrar, consultar (de forma global, filtrada por mascota, o por identificador), actualizar y dar de baja lógica las vacunas aplicadas a una mascota, restringiendo la consulta de un usuario con rol `ROLE_DUEÑO` a las vacunas de sus propias mascotas. - **Rationale:** nuevo requisito de la actividad de Unidad IV; formaliza el módulo `VacunaController/VacunaService`, sin requisito formal previo en el SRS. - **Verificación:** Test `VacunaControllerTest` (9 pruebas); colección Postman `docs/postman/BIOPET-Vacunas.postman_collection.json` (12 requests, ejecutable con Newman). La operación GET `/api/vacunas/mascota/{mascotaId}` no tiene prueba automatizada dedicada en `VacunaControllerTest`; las demás operaciones sí. - **Trazabilidad:** → HU-023 → CU-23 → `VacunaController.{listar,listarPorMascota,buscar,crear,actualizar,eliminar}` → `VacunaService` → GET `/api/vacunas`, GET `/api/vacunas/mascota/{mascotaId}`, GET `/api/vacunas/{id}`, POST `/api/vacunas`, PUT `/api/vacunas/{id}`, DELETE `/api/vacunas/{id}`. - **Estado:** verificado.

REQ-F-025 — Consulta de información externa de especies - Tipo: Funcional · **Prioridad:** Could - **Enunciado:** El sistema deberá permitir consultar información de una especie (nombre científico, hábitat, dieta) desde un servicio externo, utilizando una caché Redis para evitar llamadas repetidas al servicio externo dentro de una ventana de tiempo configurable. - **Rationale:** nuevo requisito de la actividad de Unidad IV; integración con un proveedor externo (API Ninjas) vía `ExternalApiClient`, con patrón *cache-aside* en `ExternalApiService` (TTL configurable mediante `app.external-api.cache-ttl-seconds`, valor por defecto 600 s). - **Verificación:** Test `ExternalApiServiceTest` (6 pruebas, `Backend/src/test/java/com/biopet/integration/`) — corrige una afirmación de una versión anterior de este documento, que declaraba “sin prueba auto-

matizada dedicada”; esa clase de test sí existe en el código actual. Evidencia empírica adicional en docs/u4/evidencias/fred/redis-cache-comparacion.md (comparación real de tiempos de respuesta con caché fría vs. caché caliente). - **Trazabilidad:** → HU-024 → CU-24 → ExternalApiController.infoEspecie → ExternalApiService.obtenerInfoEspecie → ExternalApiClient → GET /api/externa/especies. - **Estado:** verificado.

3.2. Requisitos no funcionales (REQ-NF)

REQ-NF-001 — Rendimiento del listado de mascotas - Categoría: Rendimiento · **Prioridad:** Must - **Enunciado:** El tiempo de respuesta del listado de mascotas no deberá superar 200 ms (p95) con caché caliente ni 500 ms (p95) con caché fría. - **Rationale:** heredado de RNF-01/RNF-WEB-04 de la Entrega 1A, con umbrales cuantitativos añadidos por el bloque C.1 de la Guía. - **Verificación:** k6, 5 corridas en caliente y 5 en frío sobre el tag v1.0.0, docs/mediciones/perf/ (archivos k6-20260903T*-local-tls-v1.0.0-*.json y REPORT.md). - **Estado:** verificado. docs/mediciones/perf/REPORT.md (2026-09-03) registra p95 entre 6.04 ms y 18.56 ms según la corrida (máximo 15.06 ms en caché caliente y 18.56 ms en caché fría), muy por debajo de los umbrales de 200 ms (caché caliente) y 500 ms (caché fría) exigidos por este requisito, con 0.0 % de error en todas las corridas.

REQ-NF-002 — Comunicación cifrada obligatoria - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** Toda comunicación entre cliente y servidor deberá realizarse mediante HTTPS/TLS. - **Rationale:** heredado de RNF-02/RNF-WEB-02. - **Verificación:** curl -v mostrando TLSv1.3 y suite AEAD; SecurityHeadersTest. - **Trazabilidad:** docker-compose.tls.yml, application-tls.yml, scripts/generate-dev-keystore.sh. - **Estado:** verificado en configuración TLS de desarrollo.

REQ-NF-003 — Expiración configurable de tokens JWT - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** El sistema deberá emitir access tokens con expiración de 1 hora y refresh tokens con expiración de 7 días, ambos configurables mediante variables de entorno (JWT_EXPIRATION_MS, JWT_REFRESH_EXPIRATION_MS), sin valores fijos en el código. - **Rationale:** heredado de RNF-03/RNF-WEB-03. - **Verificación:** JwtServiceTest; inspección de application.yml. - **Estado:** verificado.

REQ-NF-004 — Claims estándar del JWT - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** Cada JWT emitido deberá incluir los siete claims estándar (iss, sub, aud, exp, nbf, iat, jti) conforme al RFC 7519. - **Rationale:** refinamiento de RNF-03 exigido por el bloque A.1 de la Guía; verificado directamente en JwtService.java (issuer, audience configurables vía JWT_ISSUER/JWT_AUDIENCE). - **Verificación:** JwtServiceTest. - **Estado:** verificado.

REQ-NF-005 — Interfaz responsiva - Categoría: Usabilidad · **Prioridad:** Must - **Enunciado:** La interfaz deberá adaptarse correctamente a resoluciones entre 320px y 1440px (móvil, tablet, escritorio). - **Rationale:** heredado de RNF-04/RNF-WEB-01. - **Verificación:** Lighthouse (perfil móvil Slow 4G + perfil desktop) + inspección manual en Chrome DevTools a 320px, 768px y 1440px. - **Trazabilidad:** frontend/src/app/features/mascotas.component.ts (.grid-mascotas con @media (max-width: 600px)). - **Estado:** verificado. docs/mediciones/lighthouse/ contiene 6 corridas originales (2026-08-01, perfil móvil) más 12 corridas de re-ejecución (2026-08-18, perfiles móvil y desktop, /login y /mascotas, lhci-20260818-0538-*.json). Performance ≥90 en todos los casos (100 en desktop, 90-94 en móvil), Accessibility 91 en todos los casos, en ambos perfiles y ambas rutas (incluida la vista autenticada de Mascotas).

REQ-NF-006 — Compatibilidad de navegadores - Categoría: Compatibilidad · **Prioridad:** Should - **Enunciado:** La aplicación deberá funcionar correctamente en las versiones modernas de Chrome, Firefox y Edge. - **Rationale:** heredado de RNF-05/RNF-WEB-05. - **Verificación:** ejecución manual de los flujos críticos en los tres navegadores. - **Estado:** pendiente de evidencia archivada.

REQ-NF-007 — Disponibilidad durante evaluaciones académicas - Categoría: Disponibilidad · **Prioridad:** Must - **Enunciado:** El sistema deberá estar operativo durante las semanas de evaluación establecidas por la asignatura. - **Rationale:** heredado de RNF-06. - **Verificación:** demostración en vivo (make up) durante la semana de entrega; en producción, healthcheck de Render (docs/despliegue/). - **Estado:** pendiente de confirmación explícita (Must #1 de 2, ver sección 7). La afirmación “verificado por diseño” de una versión anterior de este documento no está respaldada por un artefacto concreto (log de

uptime, captura de healthcheck con fecha, o corrida de monitoreo real); es una inferencia razonable, no una verificación empírica — y coincide con lo que ya registra `docs/trazabilidad/matriz.csv` para esta misma fila: estado “implementado” (no “verificado”), con la nota explícita “no existe evidencia de disponibilidad sostenida durante una semana completa de evaluación”. Se solicitó a Fred (responsable de despliegue/Render) el nombre del artefacto o commit que la respalde antes de volver a marcarla “verificado”.

REQ-NF-008 — Cifrado de contraseñas - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** Las contraseñas de los usuarios deberán almacenarse cifradas mediante un algoritmo hash seguro (BCrypt), nunca en texto plano. - **Rationale:** heredado de RNF-07. - **Verificación:** `AuthControllerTest`; inspección de `db/seed.sql` (hash BCRYPT del usuario admin) y de `SecurityConfig` (`PasswordEncoder`). - **Estado:** verificado.

REQ-NF-009 — Registro de eventos de autenticación (OWASP A09) - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** El sistema deberá registrar cada evento de autenticación (login exitoso, login fallido, bloqueo por rate limit, refresh, logout) con IP, marca de tiempo y sujeto (correo), sin exceder 200 caracteres por campo registrado. - **Rationale:** requisito nuevo derivado del control OWASP A09, exigido por el bloque C.2 de la Guía; ausente en el SRS original. Verificado directamente en `AuthenticationAuditService.java`. - **Verificación:** `AuthenticationAuditServiceTest`; captura de log real en `docs/mediciones/sec/A09-logging.md`. - **Estado:** verificado. `docs/mediciones/sec/A09-logging.md` incluye líneas reales de log (`AUTH_AUDIT event=LOGIN_SUCCESS, LOGIN_FAILURE, LOGIN_RATE_LIMITED`, con IP, timestamp UTC y sujeto) y documenta la sanitización contra log forging.

REQ-NF-010 — Limitación de intentos de login (OWASP A07) - Categoría: Seguridad · **Prioridad:** Must - **Enunciado:** El sistema deberá bloquear temporalmente los intentos de login desde una misma IP tras 6 intentos fallidos consecutivos dentro de una ventana de 15 minutos, respondiendo 429 Too Many Requests durante el bloqueo, con parámetros configurables mediante variables de entorno. - **Rationale:** requisito nuevo derivado del control OWASP A07, exigido por el bloque C.2 de la Guía; ausente en el SRS original. Verificado directamente en `LoginRateLimiterService.java` (`security.rate-limit.login.max-attempts, .window, .block-duration`). - **Verificación:** `LoginRateLimiterServiceTest`; evidencia HTTP real en `docs/mediciones/sec/A07-authentication.md`. - **Estado:** verificado. `docs/mediciones/sec/A07-authentication.md` registra los 5 primeros fallos respondiendo 401, el 6º respondiendo 429 con `ProblemDetail` (`type=urn:biopet:error:rate-limit`) y cabecera `Retry-After: 900` real, capturada con `curl`, no solo con la prueba JUnit.

REQ-NF-011 — Arquitectura en capas - Categoría: Mantenibilidad · **Prioridad:** Must - **Enunciado:** La arquitectura del backend deberá organizarse en capas separadas de presentación (`controller`), lógica de negocio (`service`) y acceso a datos (`repository/entity`). - **Rationale:** heredado de RNF-08. - **Verificación:** revisión de la estructura de paquetes `com.biopet.{controller,service,repository,entity,dto}`. - **Estado:** verificado.

REQ-NF-012 — Caché del listado de mascotas con TTL externo - Categoría: Rendimiento · **Prioridad:** Should - **Enunciado:** El listado paginado de mascotas deberá almacenarse en caché Redis con un tiempo de vida (TTL) configurable mediante variable de entorno (`CACHE_TTL_MS`, valor por defecto 300000 ms), sin valores fijos en código, y su tasa de aciertos (hit ratio) deberá medirse y reportarse empíricamente. - **Rationale:** heredado de la estrategia de caché ya definida en `ADR-003-jwt-redis.md`; llevado a requisito no funcional explícito exigido por el bloque A.1 de la Guía. - **Verificación:** inspección de `application.yml` (`spring.cache.redis.time-to-live`); evidencia cruda en `docs/mediciones/redis/`. - **Estado:** verificado parcialmente. `docs/mediciones/redis/` confirma el TTL real de la clave de caché (195 s, mediante TTL) y su existencia bajo carga (`DBSIZE, KEYS mascotas:.*`). No hay todavía una medición explícita de hit ratio (aciertos/total); solo se verificó que la clave existe con el TTL configurado, no su tasa de aciertos a lo largo del tiempo. Esto queda pendiente (ver Observaciones, sección 7).

REQ-NF-013 — Estrategia híbrida de acceso a datos (ORM + procedimientos almacenados) - Categoría: Mantenibilidad / Seguridad · **Prioridad:** Must - **Enunciado:** Toda operación de base de datos que no sea un CRUD elemental sobre una única tabla (según la definición del bloque A.2.1 de la Guía) deberá implementarse como función o procedimiento almacenado versionado en `db/procs/`, invocado desde un repositorio Spring Data, quedando prohibida la concatenación de entrada de usuario en cualquier

fragmento JPQL, HQL o SQL nativo. - **Rationale:** requisito nuevo exigido explícitamente por el bloque A.2 de la Guía. Se marcó “cumplida por alcance actual” en una versión anterior de este documento porque en ese momento no existía ninguna operación no elemental; en v0.9.0-rc ya existe una implementación real: `fn_resumen_mascotas_por_especie` (ver REQ-F-021). - **Verificación:** `ResumenEspeciesIntegrationTest`, `ProcedimientosBiopetIntegrationTest`; `scripts/audit-sql-dynamic.sh` (ausencia de SQL dinámico por concatenación); `docs/basedatos/CATALOGO-SP.md`. - **Estado:** verificado. La confirmación pendiente en una versión anterior de este documento (que `scripts/audit-sql-dynamic.sh` se hubiera ejecutado realmente sobre el estado v1.0.0 del código, con las rutinas ya reclasificadas a `PROCEDURE`, y no solo sobre el estado v0.9.0-rc anterior) ya se realizó: la reproducción se generó **posteriormente**, sobre el commit exacto del tag histórico v1.0.0 (0d5cd525ce648cca7219da204e16fa622e671a87), en un worktree aislado que no modificó el árbol de trabajo principal — el código verificado es el del tag; el log es evidencia generada después, no un artefacto que existiera dentro del commit. Resultado: `audit-sql-dynamic: 0 hallazgos en 7 archivo(s)`. PASA (código de salida 0). Evidencia archivada en `docs/mediciones/sec/audit-sql-dynamic-v1.0.0.txt` (`docs/mediciones/sec/audit-sql-dynamic-v1.0.0.sha256`).

3.3. Criterios INVEST aplicados a las historias de usuario

Las 24 historias de usuario de `docs/requisitos/historias/HistoriasUsuario.md` se evalúan aquí explícitamente contra los seis criterios INVEST de Cohn (Independent, Negotiable, Valuable, Estimable, Small, Testable), con justificación breve y verificable contra campos que ya existen en cada historia (no se introduce información nueva para esta evaluación).

Negotiable, Valuable y Testable se cumplen en las 24 historias, sin excepción, verificado estructuralmente así:

- **Negotiable:** las 24 historias siguen el formato Connextra (“Como... quiero... para...”), que expresa intención y valor, no una solución de diseño fija; el “cómo” (campos exactos, validaciones) queda abierto a negociación en el caso de uso y en la implementación.
- **Valuable:** cada historia declara un rol beneficiario explícito (Como <rol>) y está trazada a un requisito funcional con *rationale* propio en la sección 3.1, nunca a una tarea puramente técnica sin beneficiario.
- **Testable:** las 24 historias incluyen un bloque `gherkin` con al menos un escenario **Given/When/Then** verificable; las 8 aún no implementadas (Should/Could pendientes) lo declaran explícitamente como “comportamiento esperado, no implementado”, sin fingir que ya existe evidencia de ejecución.

Independent, Estimable y Small varían por historia (justificación por fila, tomada del campo *Dependencias* y del estado real de cada historia):

HU	Independent	Estimable	Small
HU-001	Sí	Sí — implementada, esfuerzo ya observado	Nota — 2 REQ-F relacionados
HU-002	Sí	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-003	Parcial — depende de HU-002 (requiere haber iniciado sesión previamente)	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-004	Parcial — depende de HU-002	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-005	Parcial — depende de HU-002	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-006	Parcial — depende de HU-002	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F

HU	Independent	Estimable	Small
HU-007	Parcial — depende de HU-005 (control de acceso por rol)	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-008	Parcial — depende de HU-005	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-009	Parcial — depende de HU-005, HU-008	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-010	Parcial — depende de HU-005, HU-009	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-011	Parcial — depende de HU-005, HU-009	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-012	Parcial — depende de HU-007 (requiere que la mascota exista)	Parcial — hay código/tests reales para el subconjunto entregado (ver Observaciones de la historia)	Sí — 1 REQ-F
HU-013	Parcial — depende de HU-012	Parcial — estado “Pendiente”, sin artefacto entregado que medir	Sí — 1 REQ-F
HU-014	Parcial — depende de HU-007	Parcial — hay código/tests reales para el subconjunto entregado (ver Observaciones de la historia)	Sí — 1 REQ-F
HU-015	Parcial — depende de HU-007	Parcial — estado “Pendiente”, sin artefacto entregado que medir	Sí — 1 REQ-F
HU-016	Parcial — depende de HU-012 (requiere historial clínico)	Parcial — estado “Pendiente”, sin artefacto entregado que medir	Sí — 1 REQ-F
HU-017	Parcial — depende de HU-007	Parcial — estado “Pendiente”, sin artefacto entregado que medir	Sí — 1 REQ-F
HU-018	Sí	Parcial — estado “Pendiente”, sin artefacto entregado que medir	Sí — 1 REQ-F
HU-019	Parcial — depende de HU-002 (requiere usuario autenticado)	Parcial — estado “Parcial”, sin artefacto entregado que medir	Sí — 1 REQ-F
HU-020	Parcial — depende de HU-005 (control de acceso por rol), HU-007 (requiere que existan mascotas registradas)	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-021	Parcial — depende de HU-001 (registro de usuario)	Parcial — estado “Pendiente”, sin artefacto entregado que medir	Sí — 1 REQ-F
HU-022	Parcial — depende de HU-005 (control de acceso por rol)	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F

HU	Independent	Estimable	Small
HU-023	Parcial — depende de HU-007 (registro de mascota, requiere que la mascota exista)	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F
HU-024	Sí	Sí — implementada, esfuerzo ya observado	Sí — 1 REQ-F

Lectura honesta de “Independent”. La mayoría de historias depende de al menos otra (típicamente HU-002 “sesión iniciada” o HU-005/HU-007 “rol autorizado”/“mascota existente”), lo cual es normal y esperado en un sistema con autenticación y CRUD relacional: ninguna historia de gestión de un recurso es realmente independiente de “existe sesión” y “el recurso padre existe”. Se documenta la dependencia real en vez de declarar “Independent: Sí” de forma genérica sin sustento, que habría sido la alternativa más fácil pero menos honesta.

Lectura honesta de “Estimable”. Las historias ya **implementadas** (15 de 24) son estimables con certeza retrospectiva: el código y los tests existen, su esfuerzo ya fue observado. Las **parciales** (HU-012, HU-014: 2) son estimables solo para el subconjunto ya entregado — el esfuerzo del resto (calendario interactivo, vista de historial consolidado) sigue siendo una estimación no verificada. Las **pendientes** (7) no tienen ningún artefacto que permita estimar con base empírica; su tamaño es una suposición heredada de la Entrega 1A, no una medición.

Nota sobre “Small” en HU-001. Es la única historia que agrupa dos REQ-F (registro exitoso + rechazo por correo duplicado) porque ambos comparten el mismo flujo de entrada (`POST /api/auth/registro`) y serían artificialmente pequeños por separado; se mantiene como una sola historia pequeña y cohesiva, no como una violación de INVEST.

4. Trazabilidad (resumen)

La matriz completa vive en `docs/trazabilidad/matriz.csv` (bloque A.3.3 de la Guía), con la fila raíz por requisito y las columnas: `id_requisito`, `tipo`, `prioridad_moscow`, `historia_usuario`, `caso_de_uso`, `modulo_codigo`, `endpoint_api`, `prueba_automatizada`, `tipo_acceso`, `evidencia_empirica`, `estado`. Este SRS es la fuente de verdad para las columnas `id_requisito`, `tipo`, `prioridad_moscow` y `estado`; la matriz no debe declarar un requisito que no exista aquí, ni omitir ninguno de los REQ-F/REQ-NF listados en la sección 3.

Correspondencia REQ-F ↔ HU ↔ CU (para llenar `matriz.csv`):

REQ-F	HU	CU	REQ-F	HU	CU
001	HU-001	CU-01	011	HU-010	CU-10
002	HU-001	CU-01	012	HU-011	CU-11
003	HU-002	CU-02	013	HU-012	CU-12
004	HU-003	CU-03	014	HU-013	CU-13
005	HU-004	CU-04	015	HU-014	CU-14
006	HU-005	CU-05	016	HU-015	CU-15
007	HU-006	CU-06	017	HU-016	CU-16
008	HU-007	CU-07	018	HU-017	CU-17
009	HU-008	CU-08	019	HU-018	CU-18
010	HU-009	CU-09	020	HU-019	CU-19
021	HU-020	CU-20	022	HU-021	CU-21
023	HU-022	CU-22	024	HU-023	CU-23
025	HU-024	CU-24			

4.1. Trazabilidad histórica: identificadores originales → identificadores actuales (cierre de OBS-03)

La retroalimentación oficial del SGA de la Entrega 1A señaló que “los RF-WEB se remapean a RF-16/RF-17 sin matriz de trazabilidad explícita en esta entrega” (ver `docs/observaciones/OBSERVACIONES.md`, OBS-03). Hasta esta revisión, el vínculo entre los identificadores originales (RF-NN / RF-WEB-NN, Entrega 1A) y los identificadores actuales (REQ-F-NNN) solo existía disperso en el campo *Rationale* de cada requisito individual (sección 3.1). La siguiente tabla lo consolida en un único lugar explícito, sin omitir ningún requisito funcional del sistema y sin inventar ningún origen que no estuviera ya citado en este documento:

Identificador anterior (Entrega 1A)	Identificador actual	Descripción	Caso de uso	Historia	Estado
RF-01	REQ-F-001	Registro de usuario dueño de mascota	CU-01	HU-001	verificado
RF-01, RF-02	REQ-F-008	Creación de mascota asociada a un dueño existente	CU-07	HU-007	verificado
RF-02	REQ-F-009	Listado paginado de mascotas activas por propietario/rol	CU-08	HU-008	verificado
RF-02	REQ-F-011	Actualización de mascota con verificación de propiedad	CU-10	HU-010	verificado
RF-03, RF-04	REQ-F-013	Registro y consulta de historial clínico	CU-12	HU-012	parcial
RF-05	REQ-F-014	Prescripción de medicamentos	CU-13	HU-013	pendiente
RF-06	REQ-F-015	Gestión de citas veterinarias mediante calendario	CU-14	HU-014	parcial
RF-07	REQ-F-022	Notificaciones al usuario por correo electrónico (cierre de OBS-02)	CU-21	HU-021	pendiente
RF-08, RF-09	REQ-F-016	API de recepción de telemetría de dispositivos IoT	CU-15	HU-015	pendiente
RF-10	REQ-F-017	Recomendaciones clínicas informativas (redacción cerrada en OBS-04)	CU-16	HU-016	pendiente

Identificador anterior (Entrega 1A)	Identificador actual	Descripción	Caso de uso	Historia	Estado
RF-11, RF-12	REQ-F-018	Facturación digital y registro de pagos	CU-17	HU-017	pendiente
RF-13, RF-WEB-02	REQ-F-006	Control de acceso por rol (RBAC)	CU-05	HU-005	verificado
RF-14	REQ-F-019	Reportes estadísticos exportables	CU-18	HU-018	pendiente
RF-15	REQ-F-020	Auditoría de operaciones del sistema	CU-19	HU-019	parcial
RF-16, RF-WEB-01	REQ-F-003	Autenticación mediante usuario y contraseña	CU-02	HU-002	verificado
RF-17, RF-WEB-04	REQ-F-005	Cierre de sesión con revocación de token	CU-04	HU-004	verificado
RNF-03, RNF-WEB-03	REQ-F-004	Renovación de sesión (refresh)	CU-03	HU-003	verificado

Requisitos sin origen en la Entrega 1A (nuevos, agregados durante la Tercera Entrega para documentar funcionalidad ya presente en el código, sin requisito previo que cerrar): **REQ-F-002** (rechazo de correo duplicado), **REQ-F-007** (consulta del perfil propio), **REQ-F-010** (consulta de mascota por id), **REQ-F-012** (baja lógica de mascota) y **REQ-F-021** (resumen de mascotas por especie, exigido por el bloque A.2.2 de la Guía de la Tercera Entrega). Se incorporan igualmente sin origen en la Entrega 1A, durante la Unidad IV: **REQ-F-023** (gestión administrativa de usuarios), **REQ-F-024** (gestión de vacunas) y **REQ-F-025** (consulta externa de especies con caché). Ninguno de estos sustituye ni duplica un identificador **RF-NN** original.

Esta tabla es de solo lectura respecto a `docs/trazabilidad/matriz.csv`: no lo reemplaza ni lo modifica (ese archivo está fuera del alcance de los archivos autorizados para este cierre de observaciones). Queda como acción de seguimiento incorporar, en una futura revisión de `matriz.csv`, una columna equivalente de origen histórico para **REQ-F-022**.

5. Modelo de datos (referencia)

El diccionario de datos completo de las entidades implementadas (`usuarios`, `mascotas`) está en `docs/diccionario_datos.md`, sincronizado con la migración `Flyway database/migrations/V1__schema_inicial.sql`. El modelo entidad-relación conceptual completo (incluyendo las entidades pendientes: `Cita`, `Historial_Clinico`, `Dispositivo_IoT`, `Chat_Triage`, `Producto_Servicio`, `Factura`, `Detalle_Factura`, `Proveedor`, `Mercaderia`, `Detalle_Ingreso`) proviene de la Entrega 1A (`PFC_Entrega1A_BMT.pdf`, sección 6) y se conserva como visión de producto pendiente de materialización, sin duplicar aquí el DDL completo.

Diagrama entidad-relación (DER) — dos artefactos distintos, no intercambiables (cierre de OBS-05):

- `docs/diagrams/der-biopet/der-biopet.png` — renderizado generado a partir de la fuente `Graphviz der-biopet.dot`. Es un diagrama **dibujado**, no una exportación de una herramienta de modelado de base de datos.

- `docs/observaciones/evidencias/DER-BIOPET-pgAdmin-ERD-Tool.png` — exportación **real** generada desde **pgAdmin 4**, herramienta **ERD Tool**, solicitada explícitamente por la retroalimentación oficial del SGA de la Entrega 1B: “*Exportar el DER desde pgAdmin 4 (ERD Tool) como PNG de alta resolución para el informe final*” (ver `docs/observaciones/OBSERVACIONES.md`, OBS-05). PNG válido (firma de archivo verificada), 768×883 px, generado directamente sobre el esquema real de PostgreSQL, no sobre la fuente `.dot`.

6. Interfaces de usuario (referencia)

Los wireframes conceptuales (login unificado, dashboard por rol, historial clínico, gestión de citas) están documentados en la Entrega 1A (`PFC_Entrega1A_BMT.pdf`, sección 7). La interfaz realmente implementada en v1.0.0 está en `frontend/src/app/features/`: `login.component.ts` (autenticación), `mascotas.component.ts` (CRUD con paginación, formularios, confirmación de borrado, resumen por especie y accesibilidad) y, nuevo en Unidad IV, `vacunas.component.ts` (CRUD de vacunas). **Citas, consultas, gestión administrativa de usuarios y consulta externa de especies existen como API real (ver 3.1) pero sin pantalla propia todavía** — no hay componentes correspondientes en `frontend/src/app/features/`. Ninguna de estas cinco pantallas cuenta con wireframe formal actualizado — se deja como observación abierta en la sección 7.

7. Observaciones e información pendiente

Esta sección se revisó contra el estado real del repositorio (v1.0.0). Se mantiene el mismo principio que en versiones anteriores: no se marca nada como resuelto sin evidencia verificable, y no se inventa contenido para cerrar un pendiente.

Resuelto desde la versión anterior de este documento:

- **ADR de cambio de pila tecnológica** — CERRADO. `docs/adr/ADR-002-pila-tecnologica.md` existe, está en estado “Aceptado” y documenta explícitamente la migración de ASP.NET Core 8 (ADR-001, Entrega 1A) a Java 21 / Spring Boot 3.2, con contexto, evidencia verificable en `Backend/pom.xml` y alternativas consideradas. El señalamiento que traía `CAMBIOS-SRS.md` ya no aplica.
- **Evidencia empírica de rendimiento (REQ-NF-001)** — CERRADO. 10 corridas k6 reales (5 caliente, 5 frío) en `docs/mediciones/perf/`, con p95 entre 9.7 ms y 22.13 ms, muy por debajo del umbral de 200/500 ms.
- **Evidencia OWASP A07/A09 (REQ-NF-009, REQ-NF-010)** — CERRADO. `docs/mediciones/sec/A07-authent` y `A09-logging.md` contienen capturas HTTP (`curl`) y de log reales, no solo pruebas JUnit.
- **Bitácora de observaciones** (`docs/observaciones/OBSERVACIONES.md`): existe, cubre OBS-01 a OBS-15 con fuente primaria (capturas SGA) y verificación cruzada contra `git log`. OBS-13 y OBS-14 (CRedit individual y afirmación falsa sobre ausencia de historial Git en `CONTRIBUTORS.md`) quedaron cerradas al corregirse `CONTRIBUTORS.md` con acuerdo del equipo completo.

Cierre 2026-08-18 — actualización tras la re-ejecución real de Lighthouse. Contrario a lo previsto en el cierre del 2026-08-17 (que asumía que Fred no completaría sus pendientes), el equipo sí ejecutó la re-corrida de Lighthouse y Fred sí entregó su bloque de trabajos relacionados. Se actualiza el estado real:

- **Lighthouse — perfil de escritorio y re-ejecución tras el fix de SEO** — CERRADO. Re-corrida real del 2026-08-18 (`docs/mediciones/lighthouse/lhci-20260818-0538-*.json`, 12 archivos: perfil móvil y desktop, `/login` y `/mascotas`, 3 corridas cada uno). **SEO pasó de 82/100 a 100/100 en las 12 corridas**, confirmando que los dos fixes aplicados (`meta description` en `index.html`, `robots.txt` + `angular.json`) resolvían la causa raíz por completo. Performance ≥ 90 en todos los casos (100 en desktop, 90–94 en móvil); Accessibility 91 en todos. REQ-NF-005 queda verificado en ambos perfiles.
- **PRISMA (trabajos relacionados)** — CERRADO con 10 estudios. Fred entregó su bloque (F20-F21) el 2026-08-18: 3 estudios incluidos (`docs/investigacion/handoff-fred-trabajos-relacionados.md`) + 11 referencias BibTeX con DOI verificado (`handoff-fred-referencias.bib`). Total: 3 (Zaida) + 4 (Jaime) + 3 (Fred) = **10 estudios, meta de $\geq 8-9$ cumplida**. Las 24 referencias nuevas de los tres bloques ya están integradas en `docs/informe/referencias.bib` (39 entradas totales).

Cierre 2026-09-03 — revisión a v1.0.0 (recalificación).

- **Versión y firma.** Este documento pasa de v0.9.0-rc a **v1.0.0**. El PDF `SRS-v1.0.0.pdf` **no está firmado** por el docente-director a la fecha de este cierre; se envía para aprobación y la firma se incorporará, con fecha real, solo cuando se reciba.
- **INVEST — CERRADO.** Se agrega la sección 3.3, con los seis criterios aplicados explícitamente a las 24 historias de usuario (justificación por criterio, tabla verificable fila por fila).
- **Alineación de HU-012/HU-014 y REQ-F-013/REQ-F-015 con el código actual — CERRADO.** Revisión contra el repositorio real detectó que ambos requisitos seguían marcados “pendiente” pese a que la Unidad IV ya entregó `CitaController/CitaService` y `ConsultaController/ConsultaService` con CRUD completo y tests dedicados. Se corrigen a “parcial” (no “verificado”, porque el calendario interactivo y la vista de historial consolidado siguen sin implementar) en el SRS, en `HistoriasUsuario.md` y en `CasosDeUso.md`.
- **REQ-F-025 corregido — CERRADO.** Una versión anterior de este documento afirmaba que `ExternalApiController/ExternalApiService` no tenían prueba automatizada; sí la tienen (`ExternalApiServiceTest`, 6 pruebas). Se corrige el estado a “verificado”.
- **Conteo de Must — NO se declara 22/22.** De los 22 requisitos Must, **21 están verificados con evidencia concreta** (test automatizado, captura HTTP real, o corrida de medición archivada). **1 queda explícitamente “implementado” pero no verificado** (REQ-NF-007 — disponibilidad durante evaluaciones académicas: el mecanismo de healthcheck funciona y está probado, pero no existe evidencia de disponibilidad sostenida durante una semana completa de evaluación). REQ-NF-013 (estrategia híbrida de acceso a datos), que también estuvo pendiente, ya se verificó: `scripts/audit-sql-dynamic.sh` se reprodujo sobre el commit exacto del tag histórico v1.0.0 (0d5cd525ce648cca7219da204e16fa622e671a87) con resultado “0 hallazgos en 7 archivos. PASA” (evidencia archivada en `docs/mediciones/sec/audit-sql-dynamic-v1.0.0.txt`). No se marca “verificado” sin ese respaldo, aunque una versión anterior de este documento sí los daba por verificados sin evidencia suficiente.

Cierre 2026-09-07 — sincronización SRS / matriz / código antes de la firma.

- **REQ-F-007.** La verificación pasa a citar el test automatizado real (`UsuarioControllerTest.meSigueFuncionando`, `GET /api/usuarios/me`), que la matriz ya registraba; se retira la afirmación “sin test automatizado formal registrado”, que quedó desactualizada.
- **REQ-F-024.** Corrección de conteo: `VacunaControllerTest` tiene 9 pruebas, no 10.
- **Sección 4.** Se agregan a la tabla de correspondencia REQ-F-023 (HU-022/CU-22), REQ-F-024 (HU-023/CU-23) y REQ-F-025 (HU-024/CU-24), que faltaban, y a la lista de requisitos sin origen en la Entrega 1A. La sección 4.1 pasa REQ-F-013 y REQ-F-015 de “pendiente” a “parcial”, en línea con su estado en la sección 3.1 y en la matriz.
- **REQ-NF-001.** Evidencia actualizada a la corrida oficial v1.0.0 (2026-09-03): p95 entre 6.04 ms y 18.56 ms, archivos `k6-20260903T*`.
- **Matriz de trazabilidad.** Estados y notas alineados con el SRS para REQ-F-013, REQ-F-015, REQ-F-020, REQ-F-025, REQ-NF-001 y REQ-NF-012. Detalle fila por fila en `docs/requisitos/CHANGELOG-REQ.md`.

Sigue como limitación declarada, no bloqueante para el cierre de esta entrega:

- **Hit ratio de caché Redis (REQ-NF-012).** Sigue sin una medición de `keyspace_hits/keyspace_misses`; solo hay evidencia de TTL y existencia de clave bajo carga.
- **REQ-NF-006 (compatibilidad de navegadores).** Sin evidencia archivada de ejecución manual en Chrome/Firefox/Edge.
- **Wireframes de la pantalla de Mascotas y de los módulos pendientes** (historial clínico, citas, facturación): siguen siendo los de la Entrega 1A. El producto real y su documentación de arquitectura (C4, DER, SRS) sí reflejan el sistema implementado.
- **Pantallas sin interfaz propia (API real, sin UI).** Citas (calendario interactivo), historial clínico consolidado, gestión administrativa de usuarios y consulta externa de especies: las cuatro tienen backend real y verificado, pero ninguna tiene componente en `frontend/src/app/features/` todavía.
- **Aislamiento de datos incompleto en Consultas (REQ-F-013).** `ConsultaService.listar()` no filtra por propietario para `ROLE_DUEÑO`, a diferencia de `MascotaService/VacunaService` (registrado en

`docs/trazabilidad/matriz.csv`). Un dueño de mascota autenticado podría ver consultas de mascotas ajenas a través de este endpoint. Se documenta aquí para que no quede oculto; su corrección es responsabilidad del propietario del módulo (Unidad IV), fuera de la zona de archivos de esta revisión.

Ninguno de estos puntos se cierra con datos inventados. Quedan como limitaciones explícitas de la Entrega Final, consistentes con el criterio de todo este documento.

APROBACIÓN DEL DOCENTE-DIRECTOR

- **Nombre:**
- **Cargo:**
- **Firma electrónica:**
- **Fecha:**